
Problemset de Práctica 01- Repaso: Complejidad a Heaps

Prof. Víctor Racsó Galván Oyola

Esta lista no será evaluada, su único objetivo es permitir practicar los conceptos teóricos vistos en el curso hasta ahora: Notación Asintótica, Tipos de Datos Abstractos, Arreglos y Vectores, Listas Enlazadas, Pilas, Colas y Colas Dobles, Diccionarios y Hashing, Árbol Binario de Búsqueda, y Heaps. Las preguntas están agrupadas por tema, en el mismo orden en que se vieron en clase, de menor a mayor dificultad dentro de cada tema.

Complejidad y Notación Asintótica

1. ¿Qué significa que $f(n) = O(g(n))$? Da la definición, indicando qué papel cumple la constante multiplicativa c y qué significa “para n suficientemente grande”. ¿Por qué se dice que O describe una **cota superior**?
2. Clasifica cada una de las siguientes complejidades con su nombre (constante, logarítmica, lineal, cuadrática, o exponencial): $O(1)$, $O(\log n)$, $O(n)$, $O(n^2)$, $O(2^n)$.
3. Determina la complejidad, en función de n , del siguiente fragmento:

```
1 for i ← 1 to n do
2   for j ← i to n do
3     imprimir(i, j) ;
```

Justifica contando exactamente cuántas veces se ejecuta **imprimir**.

4. Determina la complejidad, en función de n , del siguiente fragmento:

```
1 for i ← 1 to n do
2   for j ← 1 to ⌊n/i⌋ do
3     imprimir(i, j) ;
```

Un compañero dice: “el ciclo interior corre *hasta* n/i , así que en el peor caso ($i = 1$) corre hasta n veces, y como eso pasa dentro de un ciclo que también corre n veces, la complejidad total es $O(n^2)$.” ¿Está en lo correcto? Justifica sumando el costo **real** de las n iteraciones del ciclo exterior (no el costo de la **peor** iteración multiplicado por n). *Ayuda:* $1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n} \approx \ln n$.

TDA, Arreglos y Vectores

5. ¿Cuál es la complejidad de acceder al elemento en la posición i de un arreglo? ¿Y la de insertar un elemento al **inicio** de un arreglo con n elementos, si hay que desplazar todo lo demás una posición?
6. Nombra al menos una operación que un **vector** soporte directamente y que un **arreglo** (de tamaño fijo) no pueda soportar sin crear un arreglo nuevo.
7. Un vector que **duplica** su capacidad cada vez que se llena logra que **push_back** sea $O(1)$ amortizado. Si en cambio, cada vez que se llena, aumentáramos la capacidad en una cantidad **fija** (por ejemplo, +10 espacios) en vez de duplicarla, ¿cuál sería el costo amortizado de **push_back** para n inserciones? Explica por qué duplicar sí funciona y sumar una constante fija no.
8. Considera la siguiente implementación de **push_back**:

Algorithm 1: PUSH_BACK(A, x)

```
1 if  $A.tamaño = A.capacidad$  then  
2   crear un arreglo nuevo  $B$  de capacidad  $2 \cdot A.capacidad$  ;  
3   copiar los  $A.tamaño$  elementos de  $A$  a  $B$  ;  
4    $A \leftarrow B$  ;  
5  $A[A.tamaño] \leftarrow x$  ;  
6  $A.tamaño \leftarrow A.tamaño + 1$  ;
```

Sumando el costo total de n llamadas a PUSH_BACK, desde un arreglo vacío de capacidad 1, y dividiéndolo entre n , demuestra que el costo amortizado es $O(1)$. *Ayuda: las copias ocurren cuando la capacidad pasa de $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow \dots$; ¿cuánto suman en total todas las copias, sobre n inserciones?*

Listas Enlazadas

9. ¿Cuál es la complejidad de acceder al k -ésimo elemento de una lista enlazada simple? ¿Por qué no se puede hacer en $O(1)$ como en un arreglo?
10. En una lista circular simple, si en vez de guardar los punteros **head** y **tail** guardas **solo tail** (el último nodo), explica por qué igual puedes acceder tanto al primer como al último elemento en $O(1)$. *Ayuda: ¿a qué nodo apunta **tail.siguiente**?*
11. ¿Cuál es la ventaja principal de una lista **doblemente** enlazada sobre una lista **simple** al momento de eliminar un nodo del cual ya tienes un puntero directo (sin necesidad de buscarlo desde la cabeza)? ¿Qué información extra necesita cada nodo, y cómo cambia la complejidad de la eliminación?
12. Diseña un algoritmo para determinar si una lista enlazada simple tiene un **ciclo** (en vez de que el último nodo apunte a nulo, en algún punto un nodo apunta de vuelta a un nodo anterior

de la misma lista). Debes lograrlo en $O(n)$ tiempo y $O(1)$ espacio extra (sin usar un conjunto de nodos visitados). *Ayuda: recorre la lista con dos punteros que avancen a distinta velocidad, uno de un nodo en un nodo y otro de dos en dos.*

Pilas, Colas y Colas Dobles

13. ¿Por qué una **pila** (y no una cola) es la estructura natural para implementar la función “deshacer” (*undo*) de un editor de texto? ¿Qué principio de acceso (LIFO o FIFO) corresponde a la forma en que un usuario espera que funcione *undo*?

14. Ejecuta, en orden, sobre una pila vacía: `push(3)`, `push(7)`, `pop()`, `push(9)`, `push(2)`, `pop()`. ¿Qué elemento estaba en el tope (el que devolvería `top()`) justo antes de cada `pop()`, y qué queda finalmente en la pila (de tope a fondo)?

15. Tienes una cola doble implementada con un **buffer circular**, como en clase: un arreglo `buf` de capacidad m , con los campos `frente` (posición del primer elemento) y `cnt` (cuántos elementos hay). Da, en función de `frente`, `cnt` y m : (a) la posición del último elemento, (b) la posición donde `push_back` escribe el elemento nuevo, y (c) el nuevo valor de `frente` después de un `push_front`. ¿Por qué en (c) hay que sumar m antes de tomar el módulo? ¿Y por qué se guarda `cnt` en lugar de un segundo índice `final`?

16. Usando **dos pilas**, diseña un algoritmo para implementar una **cola** (`enqueue/dequeue`). Describe cómo funciona cada operación en términos de `push`, `pop` y `top` sobre las pilas, y explica por qué el costo **amortizado** de `dequeue` es $O(1)$ a pesar de que, en un caso individual, podría costar $O(n)$.

Diccionarios y Hashing

17. ¿Qué es el **factor de carga** α de una tabla hash, y cómo se calcula a partir de la cantidad de elementos n y el tamaño de la tabla m ?

18. Usando el método de la división con $m = 5$, calcula $h(12)$, $h(7)$ y $h(20)$. ¿Cuáles de estas tres claves colisionan entre sí?

19. Tienes una tabla hash de tamaño $m = 7$ con **encadenamiento**, usando $h(k) = k \bmod 7$. Inserta las claves 10, 22, 31, 4, 15 en ese orden, mostrando el contenido completo de cada *bucket* al final. ¿Cuál es el factor de carga α resultante?

20. Explica la diferencia entre resolver colisiones por **encadenamiento** y por **direccionamiento abierto**: ¿qué ocurre físicamente en la tabla cuando dos claves colisionan, en cada caso? ¿Por qué con encadenamiento el factor de carga α puede ser mayor que 1, y con direccionamiento abierto no?

21. Con el método de la división, $h(k) = k \bmod m$, ¿por qué se recomienda que m sea un número **primo** y no, por ejemplo, una potencia de 2? Da un ejemplo concreto de un conjunto de claves que colisionaría mucho más si m fuera una potencia de 2.

Árbol Binario de Búsqueda

22. Si insertas las claves 1, 2, 3, 4, 5 **en ese orden** en un BST vacío (sin ningún balanceo), ¿qué forma tiene el árbol resultante? ¿Cuál es su altura, y qué le pasa a la complejidad de **BUSCAR** en el peor caso comparado con un árbol bien balanceado de los mismos 5 elementos?

23. ¿Es cierto que todo árbol binario **completo** (todos los niveles llenos, salvo quizás el último, que se llena de izquierda a derecha) con n nodos tiene altura $\Theta(\log n)$? Justifica de forma intuitiva (no hace falta una demostración formal): piensa en cuántos nodos nuevos se agregan al aumentar la altura en uno.

24. Diseña **PREDECESOR**(x) para un nodo x de un BST (el nodo con el mayor valor estrictamente menor que el de x). Considera dos casos por separado: (a) x tiene subárbol izquierdo, y (b) x no tiene subárbol izquierdo. Da la complejidad de tu algoritmo en función de la altura h del árbol.

25. Un árbol binario de búsqueda tiene recorrido **Inorder** 2, 5, 7, 9, 12, 15, 18 y recorrido **Preorder** 9, 5, 2, 7, 15, 12, 18. Reconstruye el árbol completo (describe o dibuja su estructura), explicando el proceso que seguiste.

Heaps

26. Dado el arreglo (1-indexado) $H = [1, 3, 6, 2, 9, 4]$, ¿es un **min-heap** válido? Si no lo es, identifica **todas** las posiciones que violan la propiedad de heap, indicando contra qué padre o hijo fallan.

27. Explica por qué la propiedad de heap (padre $\geq$ hijo, en un max-heap) es una condición **local** y no garantiza un orden total entre todos los elementos: da un ejemplo concreto de dos hermanos (hijos del mismo padre) donde **no** se puede saber cuál de los dos es mayor sin compararlos directamente.

28. En un heap 1-indexado de $n = 10$ elementos, ¿cuáles son los índices de las **hojas** (nodos sin hijos)? Da la fórmula general en función de n (a partir de la fórmula de $izq(i)$), y luego escribe los índices concretos para $n = 10$.

29. Tienes el arreglo (1-indexado, max-heap) $H = [3, 14, 9, 8, 7, 4, 2]$, donde los subárboles con raíz en $izq(1) = 2$ y $der(1) = 3$ ya son max-heaps válidos, pero la raíz podría violar la propiedad de heap respecto a sus hijos. Ejecuta **SIFT-DOWN**($H, 1, 7$) en su versión para max-heap (la de clase es para min-heap: solo se invierten las comparaciones), mostrando el arreglo completo después de **cada** intercambio:

Algorithm 2: SIFT-DOWN(H, i, n) — versión para max-heap

```
1 while verdadero do
2    $m \leftarrow i$  ;
3   if  $izq(i) \leq n$  y  $H[izq(i)] > H[m]$  then
4      $m \leftarrow izq(i)$  ;
5   if  $der(i) \leq n$  y  $H[der(i)] > H[m]$  then
6      $m \leftarrow der(i)$  ;
7   if  $m = i$  then
8     romper ;
9   intercambiar  $H[i]$  con  $H[m]$  ;
10   $i \leftarrow m$  ;
```

30. CONSTRUIR-HEAP (que aplica SIFT-DOWN desde $\lfloor n/2 \rfloor$ hasta 1) es más eficiente, en total, que construir el mismo heap insertando los n elementos uno por uno con INSERTAR (que aplica SIFT-UP a cada elemento nuevo). Sin repetir la demostración formal vista en clase, explica **intuitivamente** por qué la mayoría de las llamadas a SIFT-DOWN dentro de CONSTRUIR-HEAP son baratas.